

Docker networking

So far, we have been launching our containers on a single flat shared network. Although we have not talked about it yet, this means the containers we have been launching would have been able to communicate with each other without having to use any of the host networking.

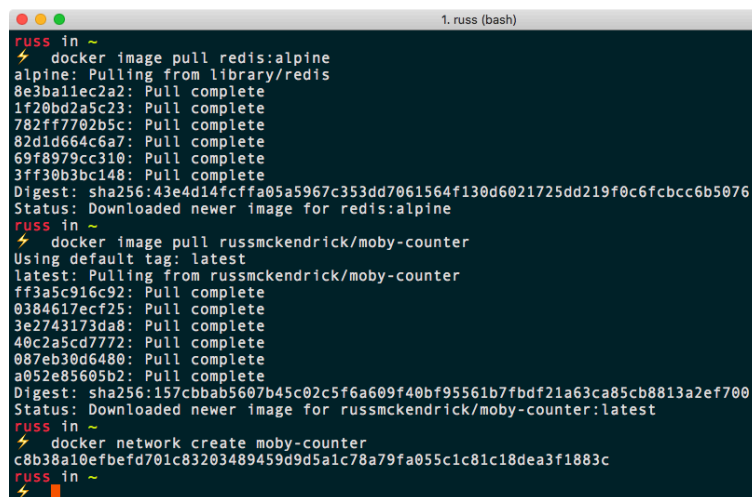
Rather than going into detail now, let's work through an example. We are going to be running a two-container application; the first container will be running Redis, and the second, our application, which uses the Redis container to store a system state.

Redis is an in-memory data structure store that can be used as a database, cache, or message broker. It supports different levels of on-disk persistence.

Before we launch our application, let's download the container images we will be using, and also create the network:

```
$ docker image pull redis:alpine
$ docker image pull russmckendrick/moby-counter
$ docker network create moby-counter
```

You should see something similar to the following Terminal output:

A terminal window titled "1. russ (bash)" showing the execution of Docker commands. The user is in a shell prompt "russ in ~". The first command is "docker image pull redis:alpine", which outputs the progress of pulling the image from the library, showing several layers being pulled and completed, and finally the digest and status. The second command is "docker image pull russmckendrick/moby-counter", which outputs the progress of pulling the latest image from the repository, showing several layers being pulled and completed, and finally the digest and status. The third command is "docker network create moby-counter", which outputs the network ID and name. The terminal text is as follows:

```
russ in ~
$ docker image pull redis:alpine
alpine: Pulling from library/redis
8e3ba11ec2a2: Pull complete
1f20bd2a5c23: Pull complete
782ff7702b5c: Pull complete
82d1d664c6a7: Pull complete
69f8979cc310: Pull complete
3ff30b3bc148: Pull complete
Digest: sha256:43e4d14fcffa05a5967c353dd7061564f130d6021725dd219f0c6fcbcc6b5076
Status: Downloaded newer image for redis:alpine
russ in ~
$ docker image pull russmckendrick/moby-counter
Using default tag: latest
latest: Pulling from russmckendrick/moby-counter
ff3a5c916c92: Pull complete
0384617ecf25: Pull complete
3e2743173da8: Pull complete
40c2a5cd7772: Pull complete
087eb30d6480: Pull complete
a052e85605b2: Pull complete
Digest: sha256:157cbbab5607b45c02c5f6a609f40bf95561b7fbd21a63ca85cb8813a2ef700
Status: Downloaded newer image for russmckendrick/moby-counter:latest
russ in ~
$ docker network create moby-counter
c8b38a10efbefd701c83203489459d9d5a1c78a79fa055c1c81c18dea3f1883c
russ in ~
```

Now that we have our images pulled and our network created, we can launch our containers, starting with the Redis one:

```
$ docker container run -d --name redis --network moby-counter redis:alpine
```

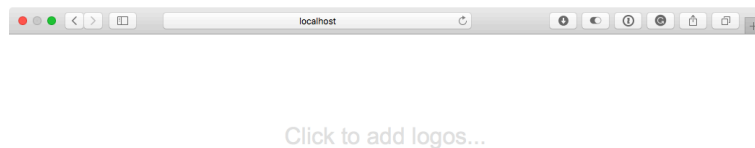
As you can see, we used the `--network` flag to define the network that our container was launched in. Now that the Redis container is launched, we can launch the application container by running the following:

```
$ docker container run -d --name moby-counter --network moby-counter -p 8080:80 rus
```

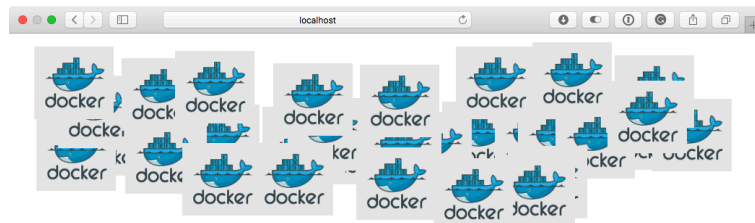
Again, we launched the container into the `moby-counter` network; this time, we mapped port `8080` to port `80` on the container. Note that we did not need to worry about exposing any ports of the Redis container. That is because the Redis image comes with some defaults that expose the default port, which is `6379` for us. This can be seen by running `docker container ls`:

```
1. russ (bash)
russ In ~
$ docker container ls
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED        STATUS
4e7931312ed2   russmckendrick/moby-counter        moby-counter  "node index.js"         About a minute ago    Up 5
8 seconds     0.0.0.0:8080->80/tcp
d760bc59c3ac   redis:alpine                       redis       "docker-entrypoint.s..." About a minute ago    Up A
bout a minute  6379/tcp
```

All that remains now is to access the application; to do this, open your browser and go to `http://localhost:8080/`. You should be greeted by a mostly blank page, with the message **Click to add logos**:



Clicking anywhere on the page will add Docker logos, so click away:



So what is happening? The application that is being served from the `moby-counter` container is making a connection to the `redis` container, and using the service to store the on-screen coordinates of each of the logos that you place on the screen by clicking.

How is the `moby-counter` application connecting to the `redis` container? Well, in the `server.js` file, the following default values are being set:

```
var port = opts.redis_port || process.env.USE_REDIS_PORT || 6379
var host = opts.redis_host || process.env.USE_REDIS_HOST || 'redis'
```

This means that the `moby-counter` application is looking to connect to a host called `redis` on port `6379`. Let's try using the `exec` command to ping the `redis` container from the `moby-counter` application and see what we get:

```
$ docker container exec moby-counter ping -c 3 redis
```

You should see something similar to the following output:

```
1. russ (bash)
russ in ~
⚡ docker container exec moby-counter ping -c 3 redis
PING redis (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.070 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.110 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.197 ms

--- redis ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.070/0.125/0.197 ms
russ in ~
⚡
```

As you can see, the moby-counter container resolves redis to the IP address of the redis container, which is 172.18.0.2. You may be thinking that the application's host file contains an entry for the redis container; let's take a look using the following command:

```
$ docker container exec moby-counter cat /etc/hosts
```

This returns the contents of /etc/hosts, which, in my case, looks like the following:

```
127.0.0.1 localhost
::1 localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
172.18.0.3 4e7931312ed2
```

Other than the entry at the end, which is actually the IP address resolving to the hostname of the local container, 4e7931312ed2 is the ID of the container; there is no sign of an entry for redis. Next, let's check /etc/resolv.conf by running the following:

```
$ docker container exec moby-counter cat /etc/resolv.conf
```

This returns what we are looking for; as you can see, we are using a local nameserver:

```
nameserver 127.0.0.11
options ndots:0
```

Let's perform a DNS lookup on redis against 127.0.0.11 using the following command:

```
$ docker container exec moby-counter nslookup redis 127.0.0.11
```

This returns the IP address of the redis container:

```
Server: 127.0.0.11
Address 1: 127.0.0.11

Name: redis
Address 1: 172.18.0.2 redis.moby-counter
```

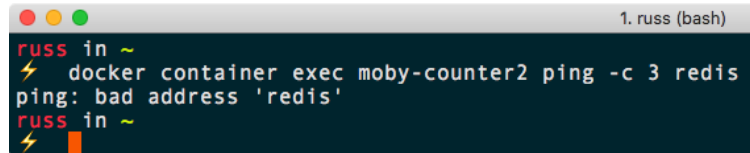
Let's create a second network and launch another application container:

```
$ docker network create moby-counter2
$ docker run -itd --name moby-counter2 --network moby-counter2 -p 9090:80 russmcken
```

Now that we have the second application container up and running, let's try pinging the redis container from it:

```
$ docker container exec moby-counter2 ping -c 3 redis
```

In my case, I get the following error:

A terminal window titled '1. russ (bash)' shows a user named 'russ' in a shell. They run the command 'docker container exec moby-counter2 ping -c 3 redis'. The output is 'ping: bad address 'redis''. The prompt then changes to 'russ in ~'.

Let's check the `resolv.conf` file to see if the same nameserver is being used already, as follows:

```
$ docker container exec moby-counter2 cat /etc/resolv.conf
```

As you can see from the following output, the nameserver is indeed in use already:

```
nameserver 127.0.0.11
options ndots:0
```

As we have launched the `moby-counter2` container in a different network to that where the container named `redis` is running, we cannot resolve the hostname of the container, so it returns a bad address error:

```
$ docker container exec moby-counter2 nslookup redis 127.0.0.11
Server: 127.0.0.11
Address 1: 127.0.0.11

nslookup: can't resolve 'redis': Name does not resolve
```

Let's look at launching a second Redis server in our second network; as we have already discussed, we cannot have two containers with the same name, so let's creatively name it `redis2`.

As our application is configured to connect to a container that resolves to `redis`, does this mean we will have to make changes to our application container? No, but Docker has you covered.

While you cannot have two containers with the same names, as we have already discovered, our second network is running completely isolated from our first network, meaning that we can still use the DNS name of `redis`. To do this, we need to add the `--network-alias` flag as follows:

```
$ docker container run -d --name redis2 --network moby-counter2 --network-alias red
```

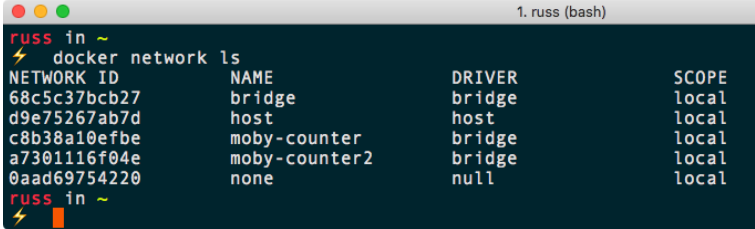
As you can see, we have named the container `redis2`, but set the `--net-work-alias` to be `redis`; this means that when we perform the lookup, we see the correct IP address returned:

```
$ docker container exec moby-counter2 nslookup redis 127.0.0.1
Server: 127.0.0.1
Address 1: 127.0.0.1 localhost

Name: redis
Address 1: 172.19.0.3 redis2.moby-counter2
```

As you can see, `redis` is actually an alias for `redis2.moby-counter2`, which then resolves to `172.19.0.3`.

Now we should have two applications running side by side in their own isolated networks on your local Docker host, accessible at `http://localhost:8080/` and `http://localhost:9090/`. Running `docker network ls` will display all of the networks configured on your Docker host, including the default networks:



```
1. russ (bash)
russ in ~
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
68c5c37bcb27        bridge             bridge              local
d9e75267ab7d        host               host                local
c8b38a10efbe        moby-counter       bridge              local
a7301116f04e        moby-counter2      bridge              local
0aad69754220        none               null                 local
```

You can find out more information about the configuration of the networks by running the following `inspect` command:

```
$ docker network inspect moby-counter
```

Running the preceding command returns the following JSON array:

```
[
  {
    "Name": "moby-counter",
    "Id": "c8b38a10efbefd701c83203489459d9d5a1c78a79fa055c1c81c18dea3f1883c",
    "Created": "2018-08-26T11:51:09.7958001Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
```

```

    "Containers": {
      "4e7931312ed299ed9132f3553e0518db79b4c36c43d36e88306aed7f6f9749d8": {
        "Name": "moby-counter",
        "EndpointID": "dc83770ae0939c98416ee69d939b30a1da391b11d14012c8188b",
        "MacAddress": "02:42:ac:12:00:03",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "d760bc59c3ac5f9ba8b7aa8e9f61fd21ce0b8982f3a85db888a5bcf103bedf6e": {
        "Name": "redis",
        "EndpointID": "5af2bfd1ce486e38a9c5cddf9e16878fdb91389cc122cfef62d5",
        "MacAddress": "02:42:ac:12:00:02",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {},
    "Labels": {}
  }
}

```

As you can see, it contains information on the network addressing being used in the IPAM section, along with details on each of the two containers running in the network.

IP address management (IPAM) is a means of planning, tracking, and managing IP addresses within the network. IPAM has both DNS and DHCP services, so each service is notified of changes in the other. For example, DHCP assigns an address to *container2*. The DNS service is then updated to return the IP address assigned by DHCP whenever a lookup is made against *container2*.

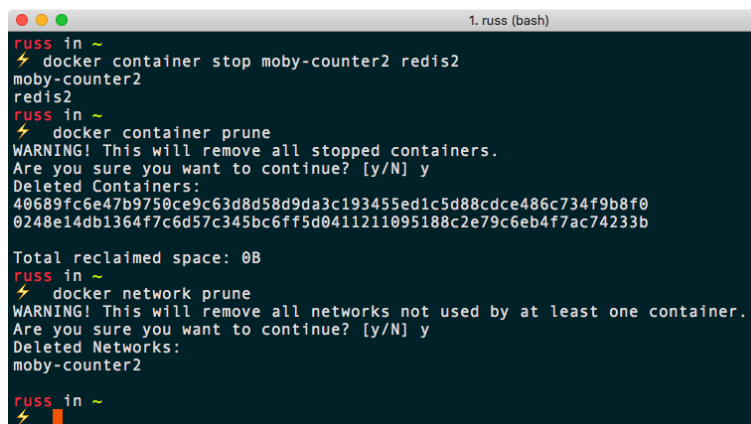
Before we progress to the next section, we should remove one of the applications and associated networks. To do this, run the following commands:

```

$ docker container stop moby-counter2 redis2
$ docker container prune
$ docker network prune

```

This will remove the containers and network, as shown in the following screenshot:



```

1. russ (bash)
russ in ~
$ docker container stop moby-counter2 redis2
moby-counter2
redis2
russ in ~
$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
40689fc6e47b9750ce9c63d8d58d9da3c193455ed1c5d88cdce486c734f9b8f0
0248e14db1364f7c6d57c345bc6ff5d0411211095188c2e79c6eb4f7ac74233b

Total reclaimed space: 0B
russ in ~
$ docker network prune
WARNING! This will remove all networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
moby-counter2

russ in ~
$

```

As mentioned at the start of this section, this is only the default network driver, meaning that we are restricted to our networks being available only on a single Docker host. In later chapters, we will look at how we can expand our Docker network across multiple hosts and even providers.

